

Java Backend Architecture

Interview Series

Chapter 19.6

AWS Data Services

50+ Interview Q&A

Code Examples

Architecture Diagrams

Advanced Topics

Java Backend Architecture Interview Series

Chapter 19.6 – AWS Data Services

Overview

DynamoDB, ElastiCache, and Aurora are the primary AWS managed data services for Java backends. DynamoDB provides serverless NoSQL at any scale; ElastiCache Redis offers sub-millisecond caching; Aurora is a high-performance managed relational DB compatible with MySQL/PostgreSQL. Understanding data modeling, read/write capacity, replication, and failover is critical for Java backend architects.

Key Definitions

Term	Definition
DynamoDB	Serverless key-value/document NoSQL. Single-digit ms latency at any scale. On-demand or provisioned capacity.
Partition Key	DynamoDB primary key attribute that determines data distribution across partitions.
Sort Key	Optional second key component enabling range queries within a partition.
GSI	Global Secondary Index: alternative access pattern with different partition/sort key. Eventual consistency.
DynamoDB Streams	Change data capture for DynamoDB. Triggers Lambda on insert/update/delete.
ElastiCache Redis	Managed Redis cluster. Sub-ms reads. Used for session caching, rate limiting, pub/sub.
ElastiCache Cluster Mode	Redis cluster with sharding + HA. Horizontal scale via shard addition.
Aurora	MySQL/PostgreSQL-compatible managed DB. 6-way replication across 3 AZs. Up to 15 read replicas.
Aurora Serverless v2	Aurora that scales compute from 0.5 to 128 ACUs (Aurora Capacity Units) per second.
Aurora Global Database	Multi-region Aurora: primary write region + up to 5 read regions. RPO ~1s, RTO <1min.

Diagram 1: DynamoDB Data Model for Java Order Service

Table: Orders

PK	SK	Attributes
USER#user123	ORDER#001	status, total, items, ts
USER#user123	ORDER#002	status, total, items, ts
USER#user456	ORDER#003	status, total, items, ts

Access patterns:
1. Get order by ID: PK=USER#userId, SK=ORDER#orderId
2. List user's orders: PK=USER#userId, SK begins_with ORDER#
3. Orders by status (GSI): GSI PK=status, SK=timestamp (all PENDING orders)

Diagram 2: ElastiCache Redis Cluster Mode

Redis Cluster Mode Enabled (3 shards, 2 replicas each):

Shard 0	Shard 1	Shard 2
Primary	Primary	Primary
slots	slots	slots
0-5460	5461-10922	10923-16383
Replica	Replica	Replica
Replica	Replica	Replica

Automatic failover: replica promoted if primary fails (< 60s)

Code Examples

DynamoDB Java (Enhanced Client):

```
// DynamoDB Enhanced Client (AWS SDK v2)
@DynamoDbBean
public class Order {
    @DynamoDbPartitionKey
    private String pk;    // USER#userId
    @DynamoDbSortKey
    private String sk;    // ORDER#orderId
    private String status;
    private BigDecimal total;
    private Instant createdAt;
    // getters/setters
}

@Repository
public class OrderRepository {
    private final DynamoDbTable<Order> table;

    // Save order
    public void save(Order order) {
        table.putItem(order);
    }

    // List user orders (query on partition key)
    public List<Order> findByUser(String userId) {
        return table.query(q -> q
            .keyConditionExpression("pk = :pk AND begins_with(sk, :prefix)")
            .expressionAttributeValues(Map.of(
                ":pk", AttributeValue.fromS("USER#" + userId),
                ":prefix", AttributeValue.fromS("ORDER#")))
            ).items().stream().collect(Collectors.toList());
    }
}
```

ElastiCache Redis in Spring Boot:

```
# application.yml
spring:
  data:
    redis:
      cluster:
        nodes:
          - clustercfg.java-cache.use1.cache.amazonaws.com:6379
      lettuce:
        cluster:
          refresh:
            adaptive: true      # auto-discover new nodes

# Service with @Cacheable
@Service
public class ProductService {
    @Cacheable(value = "products", key = "#productId",
        unless = "#result == null")
    public Product getProduct(String productId) {
        return productRepository.findById(productId).orElse(null);
    }

    @CacheEvict(value = "products", key = "#product.id")
    public void updateProduct(Product product) {
        productRepository.save(product);
    }
}
```

Aurora Connection Pool (HikariCP):

```
# application.yml - Aurora with reader endpoint
spring:
  datasource:
    url: jdbc:postgresql://aurora-cluster.cluster-abc.us-east-1.rds.amazonaws.com:5432/javadb
    username: ${DB_USER}
    password: ${DB_PASSWORD}
    hikari:
      maximum-pool-size: 20
      minimum-idle: 5
      connection-timeout: 3000
      idle-timeout: 600000
      max-lifetime: 1800000 # < Aurora idle connection timeout

# Read replica for read-only queries
second-datasource:
  url: jdbc:postgresql://aurora-cluster.cluster-ro-abc.us-east-1.rds.amazonaws.com:5432/javadb
  hikari:
    maximum-pool-size: 30 # more connections for read-heavy workload
```

Interview Q&A – AWS Data Services

Q1. How does DynamoDB partition key choice affect performance?

DynamoDB distributes data across partitions by hashing the partition key. Hot partitions occur when too many requests hit one key (user with millions of orders). Solutions: composite keys (USER#userId vs just userId), write sharding (append random suffix), DynamoDB DAX (microsecond caching). Read hot spots: ElastiCache in front of DynamoDB. Design rule: partition key should maximise cardinality and distribute access evenly.

Q2. When should you use DynamoDB on-demand vs provisioned capacity?

On-demand: pay per request, auto-scales instantly, no capacity planning. Best for: unpredictable traffic, new applications, infrequent access. Provisioned: fixed RCU/WCU, cheaper at stable predictable load. Auto Scaling adjusts provisioned capacity based on utilisation target. Cost: provisioned is 5-10x cheaper than on-demand at sustained load. Start with on-demand, switch to provisioned when traffic patterns are stable.

Q3. What is DynamoDB Streams and how do Java backends use it?

DynamoDB Streams: ordered change log of item modifications (INSERT, MODIFY, REMOVE). Retained for 24 hours. Triggers AWS Lambda: process changes in near-real-time. Java use cases: audit logging (write every change to S3/Elasticsearch), cache invalidation (evict ElastiCache entry on DynamoDB update), event sourcing (publish domain events to SNS/SQS), cross-region replication (DynamoDB Global Tables uses Streams internally).

Q4. How does ElastiCache Redis cluster mode differ from non-cluster mode?

Non-cluster mode: single shard (1 primary + up to 5 replicas). Vertical scaling only. Single primary handles all writes. Cluster mode: 1-500 shards, each with primary + replicas. Horizontal scaling: add shards for more write capacity. Data partitioned across shards (16384 hash slots). Client must be cluster-aware (Lettuce supports this). Choose non-cluster for simple caching; cluster mode for high write throughput or dataset larger than single node memory.

Q5. What is Aurora's 6-way replication and why is it fast?

Aurora separates storage from compute. A single Aurora storage volume spans 3 AZs with 2 copies per AZ = 6 copies total. Writes only need 4/6 ACKs (quorum write). Reads only need 3/6 ACKs. This means: 2 copies can fail (1 AZ down + 1 node) without data loss or unavailability. Replication is at the storage layer (no binlog replication overhead). Result: faster writes than standard RDS with better durability.

Q6. How does Aurora Serverless v2 differ from v1?

v1: scales in 5-minute intervals to predefined capacity levels, has cold start when scaling from 0. v2: scales in increments of 0.5 ACU in seconds (near-instant), supports Aurora Replicas, Global Database, Performance Insights, no cold start. v2 min: 0.5 ACU (not 0 -- always running, unlike v1). Use v2 for Java apps with variable load requiring true instant scaling without cold-start penalty. v1 best for intermittent batch workloads.

Q7. What is the Aurora reader endpoint?

Aurora provides a reader endpoint (DNS) that load-balances read-only connections across all available Aurora Replicas. Java apps configure HikariCP datasource pointing to reader endpoint for SELECT queries. Writer endpoint always routes to the primary. Reader endpoint auto-updates when replicas are added/removed or failover occurs. Spring: configure two datasources -- primary for @Transactional writes, reader for @Transactional (readOnly=true) queries.

Q8. How do you handle Aurora failover in Java applications?

Aurora failover: replica promoted to primary in 30-120s (cluster endpoint DNS updated). Java HikariCP: `connectionTimeout + keepaliveTime` detect failed connections; pool evicts stale connections. Spring retry: `@Retryable(CannotGetJdbcConnectionException.class)` retries failed DB operations. Best practice: use cluster endpoint (not instance endpoint) so DNS failover works. Set `spring.datasource.hikari.keepalive-time=30000` to detect dead connections faster than Aurora's TCP timeout.

Q9. What are DynamoDB GSIs and when are they needed?

GSI (Global Secondary Index): defines alternative access patterns with different partition and sort keys. Up to 20 GSIs per table. Separate throughput capacity. Eventual consistency (slightly behind base table). Use case: base table PK=userId, SK=orderId -- can't query all orders by status. Add GSI: PK=status, SK=createdAt to query all PENDING orders sorted by date. Sparse GSI: only items with the indexed attribute appear in GSI (saves storage and cost).

Q10. How do you implement Redis distributed locking for Java microservices?

Redisson library implements Redlock algorithm: `RLock lock = redissonClient.getLock(key); lock.lock(10, TimeUnit.SECONDS)` acquires lock with TTL (auto-expires if holder crashes). `lock.unlock()` releases. Prevents race conditions in distributed Java services: inventory deduction, scheduled job deduplication (only one pod runs job). Alternative: SET key value NX PX 10000 (atomic set-if-not-exists with expiry) via Spring Data Redis.